

# Техническое руководство: Разработка эмулятора CHIP-8 на языке C

Это руководство предназначено для студентов. Каждый этап содержит описание необходимых конструкций языка C и примеры тестов.

## 1. Типы данных и структура системы

Прежде чем писать код, необходимо подготовить «железо» в виде структур данных.

### Что сделать:

Создать структуру `Chip8`, которая будет содержать:

- **Память:** массив на 4096 байт (`unsigned char memory[4096]`).
- **Регистры:** 16 8-битных регистров общего назначения (`$V0$–$VF$`).
- **Регистр адреса (`$I$`):** 16-битный.
- **Счетчик команд (`$PC$`):** 16-битный.
- **Стек и указатель стека (`$SP$`):** массив на 16 значений и переменная для индекса.
- **Таймеры:** два 8-битных таймера (задержки и звука).
- **Экран:** массив 64x32 (обычно одномерный массив на 2048 пикселей).

Для эмулятора критически важна точность размеров данных. В языке C стандартные типы (например, `int`) могут иметь разный размер на разных системах. Поэтому мы используем заголовочный файл `<stdint.h>`.

### Конструкции C:

- `uint8_t`: для 8-битных данных (память, регистры  $V$ , таймеры).
- `uint16_t`: для 16-битных данных (регистр  $I$ ,  $PC$ , opcode).
- `struct`: для объединения всех компонентов эмулятора в одну сущность.

## Реализация

Создайте файл `chip8.h`:

```
#include <stdint.h>
```

```
typedef struct {  
    uint8_t memory[4096];  
    uint8_t V[16];      // Регистры общего назначения  
    uint16_t I;          // Индексный регистр  
    uint16_t pc;         // Счетчик команд
```

```
uint16_t stack[16];    // Стек
uint8_t sp;            // Указатель стека
uint8_t delay_timer;
uint8_t sound_timer;
uint8_t display[64 * 32]; // Состояние пикселей (0 или 1)
uint8_t keypad[16];    // Состояние клавиш
} Chip8;
```

**Тест:** Напишите функцию `init_chip8()`, которая обнуляет всё состояние, устанавливает `$PC$` в адрес `0x200` (начало программ в CHIP-8).

## 2. Загрузка программы (ROM)

На этом этапе необходимо реализовать чтение файла с диска и запись его содержимого в массив `memory` структуры `Chip8`.

### Технические требования

1. **Адрес загрузки:** Программы CHIP-8 всегда должны начинаться с адреса `0x200` (512 в десятичной системе). Область `0x000 - 0x1FF` зарезервирована (раньше там была память интерпретатора и шрифт).
2. **Режим открытия:** Файл должен открываться строго в бинарном режиме ("`rb`").
3. **Безопасность:** Необходимо проверять, поместится ли файл в оставшуюся память (4096 - 512 = 3584 байта).

### Функции стандартной библиотеки C (<stdio.h>)

- `fopen(filename, "rb")` — открывает файл.
- `fseek(fp, 0, SEEK_END)` — перемещает "курсор" в конец файла.
- `ftell(fp)` — возвращает текущую позицию курсора (так мы узнаем размер файла в байтах).
- `rewind(fp)` — возвращает курсор в начало файла.
- `fread(buffer, element_size, count, fp)` — считывает данные в массив.
- `fclose(fp)` — закрывает файл.

### Алгоритм реализации

1. Открыть файл.
2. Определить размер файла.
3. Проверить, не слишком ли он велик для памяти CHIP-8.
4. Считать данные прямо в `chip8->memory`, начиная с индекса `0x200`.

5. Закрывать файл.

#### Реализация:

```
FILE *file = fopen(filename, "rb");  
// ... проверка на NULL и определение размера ...  
fread(&chip8->memory[0x200], 1, buffer_size, file);  
fclose(file);
```

## 3. Цикл Выборки и Декодирования (Fetch-Decode)

Инструкции CHIP-8 двухбайтовые, но память однобайтовая. Нам нужно объединить два соседних байта.

#### Конструкции C:

- Побитовый сдвиг  $\ll$ : `memory[pc] << 8` перемещает байт в верхнюю часть 16-битного числа.
- Побитовое ИЛИ  $|$ : для склеивания двух частей.
- Побитовое И  $\&$ : для извлечения частей инструкции (маскирование).

#### Пример извлечения параметров:

Для опкода `0xA $I$ NNN` (установка регистра  $I$ ):

- `opcode & 0xF000` -> даст `0xA000` (тип команды).
- `opcode & 0x0FFF` -> даст `NNN` (адрес).

Для опкода `0x8XY1` (побитовое ИЛИ):

- `(opcode & 0x0F00) >> 8` -> индекс регистра  $X$ .
- `(opcode & 0x00F0) >> 4` -> индекс регистра  $Y$ .

## 4. Эмуляция инструкций (Core Logic)

Основная часть эмулятора — это гигантский switch по типу инструкции.

#### Конструкции C:

- `switch(opcode & 0xF000)`: внешний уровень для определения группы команд.
- Вложенные switch или if-else для уточнения команды.

#### Особое внимание (Арифметика):

При выполнении сложения `7XNN` или `8XY4` нужно обрабатывать переполнение.

```
uint16_t sum = chip8->V[x] + chip8->V[y];
```

```
chip8->V[0xF] = (sum > 255) ? 1 : 0; // Флаг переноса
chip8->V[x] = sum & 0xFF;
```

## 5. Графическая подсистема (SDL2)

CHIP-8 использует монохромный дисплей. Лучший способ отображения в C — библиотека **SDL2**.

### Функции SDL:

- `SDL_CreateWindow`: создание окна.
- `SDL_CreateRenderer`: создание объекта для рисования.
- `SDL_RenderPresent`: обновление картинки на экране.

### Логика DXYN (Рисование спрайта):

Это цикл в цикле. Внешний по строкам (N), внутренний по битам в байте (8 бит).

Используйте `(sprite_byte & (0x80 >> bit))` для проверки каждого бита в байте спрайта.

## 6. Ввод данных (Keypad)

Клавиатура CHIP-8 имеет 16 клавиш. Нам нужно сопоставить их с клавишами ПК.

### Конструкции C:

- `SDL_PollEvent`: проверка событий клавиатуры.
- Массив соответствия (Scancodes to Hex):

```
uint8_t key_map[16] = {
    SDL_SCANCODE_X, // 0
    SDL_SCANCODE_1, // 1
    SDL_SCANCODE_2, // 2
    // ... и так далее
};
```

## План тестирования для студентов

| Шаг           | Тест                          | Ожидаемый результат                                          |
|---------------|-------------------------------|--------------------------------------------------------------|
| Инициализация | Вызвать <code>init()</code> . | <code>pc=0x200</code> , <code>sp=0</code> , память обнулена. |

|                   |                              |                                           |
|-------------------|------------------------------|-------------------------------------------|
| <b>Память</b>     | Загрузить файл 10 байт.      | memory[0x200] до memory[0x209] заполнены. |
| <b>Fetch</b>      | В памяти 0x00, 0xE0.         | Переменная opcode должна стать 0x00E0.    |
| <b>Арифметика</b> | Выполнить 6101 (V1=1).       | Регистр V[1] должен содержать 1.          |
| <b>Цикл</b>       | Выполнить 1202 (Jump 0x202). | После цикла pc должен стать 0x202.        |
| <b>Экран</b>      | Выполнить 00E0.              | Весь массив display должен стать 0.       |

## Полезные советы:

1. **Таймеры:** Уменьшайте delay\_timer и sound\_timer, если они > 0, 60 раз в секунду. В C используйте SDL\_GetTicks() для контроля времени.
2. **Отладка:** Напишите функцию debug\_print(), которая выводит значения всех регистров и текущий опкод в консоль на каждом шаге. Это сэкономит часы поиска ошибок.